

Министерство науки и высшего образования Российской Федерации  
Пензенский государственный университет  
Кафедра «Вычислительная техника»

## **ОТЧЕТ**

по лабораторной работе №5  
по курсу «Логика и основы алгоритмизации в инженерных задачах»  
на тему «Определение характеристик графов»

Выполнили:

студенты группы 24ВВВ3

Алмакаев В.В.

Буров А.О.

Приняли:

к.т.н., доцент Юрова О.В.

к.т.н., Деев М.В.

Пенза 2025

**Цель работы** – Освоение методов программной обработки графовых структур путем реализации алгоритмов преобразования матричных представлений графов и анализа их структурных характеристик.

### Лабораторное задание:

#### Задание 1

1. Сгенерируйте (используя генератор случайных чисел) матрицу смежности для неориентированного графа  $G$ . Выведите матрицу на экран.
2. Определите размер графа  $G$ , используя матрицу смежности графа.
3. Найдите изолированные, концевые и доминирующие вершины.

#### Задание 2\*

1. Постройте для графа  $G$  матрицу инцидентности.

- 
2. Определите размер графа  $G$ , используя матрицу инцидентности графа.
  3. Найдите изолированные, концевые и доминирующие вершины.

### Код программы

C#

```
using System;

namespace GraphLab
{
    class Program
    {
        static int[,] CreateAdjacencyMatrix(int n, double edgeProbability)
        {
            int[,] matrix = new int[n, n];
            Random rand = new Random();

            for (int i = 0; i < n; i++)
            {
                for (int j = i; j < n; j++)
                {
                    double randomValue = rand.NextDouble();
                    if (randomValue < edgeProbability)
                    {
                        matrix[i, j] = 1;
                        matrix[j, i] = 1;
                    }
                }
            }
        }
    }
}
```

```

    }
    }
    }
    return matrix;
}

static void PrintMatrix(int[,] matrix, int n)
{
    Console.WriteLine($"Матрица смежности ({n}x{n}):");
    Console.Write(" ");
    for (int i = 0; i < n; i++)
    {
        Console.Write($"{i + 1,2} ");
    }
    Console.WriteLine();

    for (int i = 0; i < n; i++)
    {
        Console.Write($"{i + 1,2} ");
        for (int j = 0; j < n; j++)
        {
            Console.Write($"{matrix[i, j],2} ");
        }
        Console.WriteLine();
    }
}

static int GraphSize(int[,] matrix, int n)
{
    int edges = 0;

    for (int i = 0; i < n; i++)
    {
        for (int j = i; j < n; j++)
        {
            if (matrix[i, j] == 1)
            {
                edges++;
            }
        }
    }
    return edges;
}

static int VertexDegree(int[,] matrix, int n, int vertex)
{
    int degree = 0;

    for (int i = 0; i < n; i++)
    {
        if (matrix[vertex, i] == 1)
        {
            if (vertex == i)
            {
                degree += 2;
            }
            else
            {
                degree += 1;
            }
        }
    }
    return degree;
}

```

```

static void PrintVertexDegrees(int[,] matrix, int n)
{
    Console.WriteLine("\nСтепени вершин:");
    for (int i = 0; i < n; i++)
    {
        int degree = VertexDegree(matrix, n, i);
        Console.WriteLine($"Вершина {i + 1}: степень {degree}");
    }
}

static bool IsDominantVertex(int[,] matrix, int n, int vertex)
{
    for (int i = 0; i < n; i++)
    {
        if (i != vertex && matrix[vertex, i] != 1)
        {
            return false;
        }
    }
    return true;
}

static void FindSpecialVertices(int[,] matrix, int n)
{
    Console.WriteLine("\nСпециальные вершины:");

    Console.Write("Изолированные вершины (степень 0): ");
    bool hasIsolated = false;
    for (int i = 0; i < n; i++)
    {
        if (VertexDegree(matrix, n, i) == 0)
        {
            Console.Write($"{i + 1} ");
            hasIsolated = true;
        }
    }
    if (!hasIsolated) Console.Write("нет");
    Console.WriteLine();

    Console.Write("Концевые вершины (степень 1): ");
    bool hasEnd = false;
    for (int i = 0; i < n; i++)
    {
        if (VertexDegree(matrix, n, i) == 1)
        {
            Console.Write($"{i + 1} ");
            hasEnd = true;
        }
    }
    if (!hasEnd) Console.Write("нет");
    Console.WriteLine();

    Console.Write($"Доминирующие вершины (соединены со всеми остальными): ");

    bool hasDominant = false;
    for (int i = 0; i < n; i++)
    {
        if (IsDominantVertex(matrix, n, i))
        {
            Console.Write($"{i + 1} ");
            hasDominant = true;
        }
    }
}

```

```

    }
}
if (!hasDominant) Console.WriteLine("нет");
Console.WriteLine();
}

static void ShowMenu()
{
    Console.WriteLine("\n=== МЕНЮ ===");
    Console.WriteLine("1 - Показать матрицу смежности");
    Console.WriteLine("2 - Вывести размер графа G, изолированные, концевые и доминирующие вершины");
    Console.WriteLine("3 - Ввести новое количество вершин");
    Console.WriteLine("4 - Заккрыть программу");
    Console.WriteLine("Выберите опцию: ");
}

static void Main(string[] args)
{
    Console.OutputEncoding = System.Text.Encoding.UTF8;
    Random rand = new Random();

    int n = 0;
    int[,] adjMatrix = null;
    double edgeProbability = 0.0;

    Console.WriteLine("\n=== Генератор неориентированного графа ===\n");

    while (true)
    {
        Console.WriteLine("Введите количество вершин: ");
        if (int.TryParse(Console.ReadLine(), out n) && n > 0)
        {
            break;
        }
        Console.WriteLine("Ошибка: количество вершин должно быть положительным числом.");
    }

    while (true)
    {
        Console.WriteLine("Введите вероятность ребра (от 0.1 до 0.9): ");
        if (double.TryParse(Console.ReadLine(), out edgeProbability) && edgeProbability >= 0.1 && edgeProbability <= 0.9)
        {
            break;
        }
        Console.WriteLine("Ошибка: вероятность должна быть числом от 0.1 до 0.9.");
    }
    Console.WriteLine($"Сгенерированная вероятность ребра: {edgeProbability:F2} ({(edgeProbability * 100):F0}%");

    adjMatrix = CreateAdjacencyMatrix(n, edgeProbability);

    int choice;
    do
    {
        ShowMenu();
        if (int.TryParse(Console.ReadLine(), out choice))
        {
            switch (choice)
            {

```

```

        case 1:
            Console.WriteLine();
            PrintMatrix(adjMatrix, n);
            break;

        case 2:
            Console.WriteLine();
            int graphEdges = GraphSize(adjMatrix, n);
            Console.WriteLine($"Размер графа G: |E(G)| =
{graphEdges} ребер");

            PrintVertexDegrees(adjMatrix, n);
            FindSpecialVertices(adjMatrix, n);
            break;

        case 3:
            while (true)
            {
                Console.WriteLine("\nВведите новое количество вершин:
");
                if (int.TryParse(Console.ReadLine(), out n) && n >
0)
                {
                    break;
                }
                Console.WriteLine("Ошибка: количество вершин должно
быть положительным числом.");
            }

            while (true)
            {
                Console.WriteLine("Введите вероятность ребра (от 0.1 до
0.9): ");
                if (double.TryParse(Console.ReadLine(), out edge-
Probability) &&
                    edgeProbability >= 0.1 && edgeProbability <=
0.9)
                {
                    break;
                }

                Console.WriteLine("Ошибка: вероятность должна быть
числом от 0.1 до 0.9.");
            }
            Console.WriteLine($"Сгенерированная вероятность ребра:
{edgeProbability:F2} ({(edgeProbability * 100):F0}%");
            adjMatrix = CreateAdjacencyMatrix(n, edgeProbability);
            Console.WriteLine("Новый граф создан успешно!");
            break;

        case 4:
            Console.WriteLine("\nПрограмма завершена...");
            break;

        default:
            Console.WriteLine("Неверный выбор! Пожалуйста, выберите
опцию от 1 до 4.");
            break;
    }
}
else
{
    Console.WriteLine("Неверный ввод! Пожалуйста, введите число от 1
до 4.");
}

```

```
        }  
    } while (choice != 4);  
}  
}
```

## **Результаты работы программы**

```

=== Генератор неориентированного графа ===

Введите количество вершин: 5
Введите вероятность ребра (от 0.1 до 0.9): 0,6
Сгенерированная вероятность ребра: 0,60 (60%)

=== МЕНЮ ===
1 - Показать матрицу смежности
2 - Вывести размер графа G, изолированные, концевые и доминирующие вершины
3 - Ввести новое количество вершин
4 - Заккрыть программу
Выберите опцию: 2

Размер графа G:  $|E(G)| = 6$  ребер

Степени вершин:
Вершина 1: степень 0
Вершина 2: степень 3
Вершина 3: степень 3
Вершина 4: степень 2
Вершина 5: степень 4

Специальные вершины:
Изолированные вершины (степень 0): 1
Концевые вершины (степень 1): нет
Доминирующие вершины (соединены со всеми остальными): нет

=== МЕНЮ ===
1 - Показать матрицу смежности
2 - Вывести размер графа G, изолированные, концевые и доминирующие вершины
3 - Ввести новое количество вершин
4 - Заккрыть программу
Выберите опцию: 1

Матрица смежности (5x5):
    1  2  3  4  5
1  0  0  0  0  0
2  0  0  1  1  1
3  0  1  0  1  1
4  0  1  1  0  0
5  0  1  1  0  1

=== МЕНЮ ===
1 - Показать матрицу смежности
2 - Вывести размер графа G, изолированные, концевые и доминирующие вершины
3 - Ввести новое количество вершин
4 - Заккрыть программу
Выберите опцию: █

```

Рисунок 1 - Результат работы программы



**Вывод:** В результате выполнения работы были успешно реализованы алгоритмы преобразования матрицы смежности в матрицу инцидентности и разработаны методы анализа вершин графа.